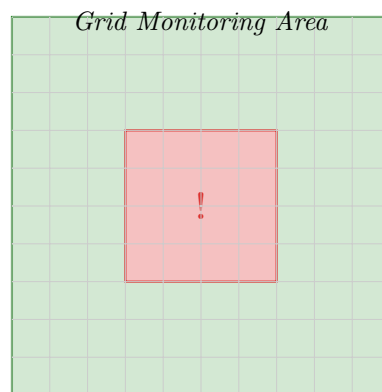


Deforest.id

Sistem Monitoring Kerusakan Hutan
Berbasis Grid & Early Warning System Terotomatisasi



Tim [Nama Tim]

[Nama-nama Anggota Tim]

Proposal Hackathon – 30 Mei 2026

"Dari Pasif Menjadi Proaktif: Mengawal Hutan Indonesia dengan Teknologi"

Daftar Isi

1	Abstrak Eksekutif	2
2	Latar Belakang & Problem Statement	2
2.1	Fakta Deforestasi di Indonesia	2
2.2	Masalah Sistem Monitoring Saat Ini	2
2.3	Pertanyaan Kunci	2
3	Solusi & Unique Selling Proposition	2
3.1	Apa Itu Deforest.id?	3
3.2	Unique Selling Proposition (USP)	3
4	Arsitektur Sistem	3
4.1	High-Level Architecture	3
4.2	Alur Data End-to-End	3
5	Tech Stack & Resource Allocation	4
5.1	Pemilihan Teknologi	4
5.2	Resource Allocation di Proxmox	5
5.3	Strategi Anti-Bottleneck	5
6	Database Schema	5
6.1	Entity Relationship Diagram	5
6.2	DDL Highlights	6
6.3	Query Contoh untuk Dashboard	7
7	User Interface & User Experience	7
7.1	Dashboard Layout	8
7.2	Wireframe (Deskripsi)	8
7.3	Format Notifikasi WhatsApp	8
8	Roadmap Pengembangan	8
9	Risk Management	9
10	Success Metrics	10
11	Rencana & Struktur Tim	10
11.1	Pembagian Peran	10
12	Potensi Pengembangan ke Depan	10
13	Penutup	11

Abstrak Eksekutif

Indonesia adalah negara dengan hutan tropis terbesar ketiga di dunia, namun juga memiliki tingkat deforestasi tertinggi secara global. Sistem pemantauan yang ada saat ini bersifat **pasif dan reaktif**—data citra satelit tersedia, tetapi tidak ada mekanisme peringatan dini yang otomatis dan langsung menjangkau petugas di lapangan.

Deforest.id hadir sebagai solusi: sebuah **Early Warning System (EWS)** yang mengubah pemantauan hutan dari pasif menjadi proaktif. Sistem ini secara otomatis menarik citra satelit dari **Google Earth Engine**, mendeteksi kerusakan lahan menggunakan **YOLOv8** pada grid-grid koordinat terstandarisasi, menampilkan hasil dalam **dashboard interaktif berbasis peta** (Leaflet.js) dengan kode warna merah → kuning → hijau, dan mengirim **notifikasi WhatsApp otomatis** berisi koordinat persis dan gambar ke petugas lapangan.

Dengan arsitektur **distributed container** (Docker di atas Proxmox), beban komputasi Machine Learning diisolasi dari backend, memastikan sistem tetap responsif meskipun sedang memproses ribuan grid citra satelit.

Latar Belakang & Problem Statement

Fakta Deforestasi di Indonesia

- Indonesia kehilangan **1,6 juta hektar** hutan per tahun (Global Forest Watch, 2023)
- 72% deforestasi disebabkan oleh perluasan lahan kelapa sawit dan pertambangan ilegal
- Kerugian ekonomi akibat deforestasi diperkirakan mencapai **Rp 87 triliun/tahun**
- Emisi karbon dari deforestasi menyumbang 15% dari total emisi nasional

Masalah Sistem Monitoring Saat Ini

1. **Passive Monitoring** — Data satelit tersedia tetapi tidak ada mekanisme peringatan dini. Staf harus *actively check* platform seperti Global Forest Watch.
2. **Time Lag** — Dari akuisisi data hingga tindak lanjut di lapangan bisa memakan **waktu berminggu-minggu**.
3. **Akses Terbatas** — Dashboard yang ada terlalu kompleks untuk petugas lapangan yang mungkin memiliki keterbatasan akses internet dan perangkat.
4. **Tidak Ada Spesifisitas** — Data yang tersedia seringkali terlalu umum, tidak memberikan koordinat persis area yang terdampak.

Pertanyaan Kunci

"Bagaimana membangun sistem yang secara otomatis mendeteksi, melokalisasi, dan memberitahukan kerusakan hutan dalam hitungan menit — bukan minggu — dengan biaya minimal dan tanpa memerlukan keahlian teknis dari pengguna akhir?"

Solusi & Unique Selling Proposition

Apa Itu Deforest.id?

Deforest.id adalah platform **Early Warning System (EWS)** terintegrasi yang mengotomatisasi seluruh pipeline dari akuisisi data satelit hingga notifikasi ke petugas lapangan, menggunakan grid-based detection dan Machine Learning.

Unique Selling Proposition (USP)

USP 1: Proaktif, Bukan Reaktif

Sistem secara otomatis menjadwalkan pull data satelit, menjalankan deteksi ML, dan mengirim notifikasi — tanpa perlu intervensi manual. Respon bisa turun dari **minggu ke menit**.

USP 2: Grid-Based Precision

Wilayah hutan dibagi ke dalam grid-grid koordinat terstandarisasi (misal: $256m \times 256m$). Setiap grid diproses secara independen, memungkinkan deteksi hingga level **sub-hektar** dengan koordinat presisi yang bisa langsung digunakan untuk tindak lanjut.

USP 3: Multi-Platform Accessibility

- **Dashboard Web** (Leaflet.js) — Peta interaktif dengan visualisasi grid warna merah/kuning/hijau
- **WhatsApp Bot** (Baileys) — Notifikasi langsung ke smartphone, tanpa perlu aplikasi tambahan
- **Detail Modal** — Side-by-side perbandingan citra asli vs annotated dengan bounding box

USP 4: Distributed & Scalable Architecture

Setiap komponen (Data Fetcher, ML Inference, Backend API, Database, Bot) berjalan di **container terpisah** dengan resource alokasi independen. ML yang berat tidak akan membebani backend API.

USP 5: Zero Manual Intervention

Seluruh pipeline dari **aku-sisi data** → **deteksi** → **notifikasi** berjalan otomatis. Cocok untuk daerah dengan SDM terbatas.

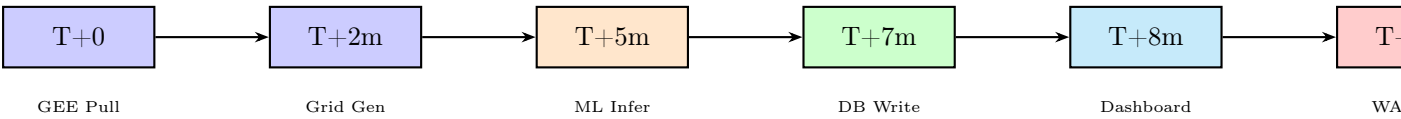
Arsitektur Sistem

High-Level Architecture

Alur Data End-to-End

1. **Akuisisi Data** — GEE Fetcher (Python) menarik citra Landsat 8/9 atau Sentinel-2 secara periodik (setiap 6 jam, *configurable*).
2. **Grid Generation** — Area of Interest (AOI) dibagi menjadi grid-grid berukuran tetap ($256m \times 256m$). Setiap grid memiliki ID unik, centroid (lat/lng), dan polygon geometry yang disimpan di PostGIS.
3. **Image Cropping** — Setiap grid dipotong (*clipped*) dari citra besar dan disimpan sebagai file PNG terpisah.

- 4. **ML Inference** — Queue-driven ML Worker (YOLOv8) memproses grid images dalam batch. Output: confidence score, damage category (severe/moderate/mild/none), dan bounding boxes.
- 5. **Database Write** — Hasil deteksi disimpan ke tabel `detection_logs`. Status grid di-update. Trigger database otomatis membuat alert untuk deteksi dengan confidence > 70%.
- 6. **Realtime Push** — Backend API (Bun + Elysia) mengekspos REST endpoint dan WebSocket. Dashboard menerima update real-time.
- 7. **Notifikasi** — WA Bot melakukan polling setiap 30 detik ke tabel `alerts`. Jika ada alert baru yang belum terkirim, bot mengirim pesan WhatsApp berisi koordinat, confidence score, dan gambar annotated.



Gambar 1: Timeline End-to-End Pipeline (target: < 15 menit)

Tech Stack & Resource Allocation

Pemilihan Teknologi

Komponen	Teknologi	Alasan
Data Source	Google Earth Engine API	Akses gratis ke Landsat/Sentinel-2, cloud masking built-in
ML Framework	YOLOv8 (Ultralytics)	State-of-the-art object detection, ringan, inference cepat
ML Optimization	TensorFlow Lite / ONNX	Deployment CPU-friendly, model size ~5–10MB
Backend Runtime	Bun + Elysia.js	3× lebih cepat dari Node.js, native TS, memori rendah
Database	PostgreSQL 16 + PostGIS 3.4	Spatial indexing (R-Tree), query geometry, robust
Cache & Queue	Redis 7	Pub/Sub untuk WebSocket, antrian ML job, fast cache
Frontend	React 18 + Vite + TypeScript	Build cepat, HMR, ecosystem matang
Map Library	Leaflet.js 1.9	Open-source, tanpa API key, plugin kaya
Visualisasi	D3.js	Fleksibel untuk time-series chart
WA Bot	Baileys (Node.js)	Library WA Web gratis, komunitas aktif
Container	Docker + Compose	Isolasi resource, reproducible deployment
Reverse Proxy	Nginx (Alpine)	Load balancing, SSL termination, static serving

Resource Allocation di Proxmox

Asumsi spesifikasi server: **16 vCPU, 32 GB RAM, 500 GB SSD.**

Container	CPU	RAM	Storage	Catatan
GEE Fetcher	1	2 GB	50 GB	Periodic job
ML Inference	4	8 GB	20 GB	Bottleneck utama
Backend API	1	1 GB	1 GB	Serving requests
PostgreSQL	+	4 GB	100 GB	I/O intensive
PostGIS				
Frontend	1	1 GB	1 GB	Static files
WA Bot	1	512 MB	1 GB	Event-based
Redis	1	1 GB	5 GB	Cache + Pub/Sub
Nginx	0.5	256 MB	500 MB	Reverse proxy
Total	11.5	17.8 GB	178.5 GB	

Sisa resource (~ 4.5 vCPU, 14 GB RAM) untuk overhead Proxmox, monitoring, dan buffer scaling.

Strategi Anti-Bottleneck

1. **CPU Pinning** — Container ML di-pin ke CPU core terpisah (fisik, bukan virtual) agar tidak berebut resource dengan backend API.
2. **ML Batch Processing** — ML memproses grid dalam batch (100 grid/siklus), tidak perlu real-time.
3. **Redis Queue** — GEE Fetcher push job ke Redis, ML Worker consume dari queue. Backend tidak terblokir.
4. **Async Architecture** — Backend API tidak menunggu hasil ML. ML menulis langsung ke DB, WebSocket push ke client.
5. **Connection Pooling** — Backend menggunakan koneksi pool ke PostgreSQL, bukan koneksi per request.

Database Schema

Entity Relationship Diagram

Berikut adalah struktur tabel utama dalam sistem Deforest.id:

- **grid_cells** — Menyimpan grid koordinat spasial dengan geometry polygon dan centroid. Setiap grid memiliki **status** (healthy/mild/moderate/severe/unknown) yang di-update otomatis oleh trigger saat ada deteksi baru.
- **satellite_imagery** — Metadata setiap citra satelit yang ditarik dari GEE: source, scene ID, cloud cover, bounds geometry.
- **detection_logs** — Hasil deteksi ML per grid: confidence score, damage category, bounding boxes (JSONB), path ke annotated image.

- **alerts** — Peringatan yang di-generate otomatis oleh trigger database saat deteksi dengan confidence ≥ 0.7 .
- **notification_logs** — Audit trail pengiriman notifikasi (channel, recipient, status, retry count).
- **config** — Key-value store untuk konfigurasi dinamis (threshold, interval, recipient list) tanpa perlu rebuild container.

DDL Highlights

Berikut adalah potongan DDL untuk tabel inti:

Listing 1: DDL Tabel `grid_cells` dan `detection_logs`

```

1  -- ENUM types
2  CREATE TYPE grid_status AS ENUM
3      ('healthy', 'mild', 'moderate', 'severe', 'unknown');
4  CREATE TYPE damage_category AS ENUM
5      ('severe', 'moderate', 'mild', 'none');
6
7  -- Core spatial table
8  CREATE TABLE grid_cells (
9      id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
10     grid_code   VARCHAR(32) UNIQUE NOT NULL,
11     geometry    GEOMETRY(POLYGON, 4326) NOT NULL,
12     centroid    GEOGRAPHY(POINT, 4326) NOT NULL,
13     area_ha     NUMERIC(10, 4),
14     status      grid_status DEFAULT 'unknown',
15     metadata    JSONB DEFAULT '{}',
16     created_at  TIMESTAMPTZ DEFAULT NOW(),
17     updated_at  TIMESTAMPTZ DEFAULT NOW()
18 );
19
20 -- Spatial index for fast bounding-box queries
21 CREATE INDEX idx_grid_geometry_gist
22     ON grid_cells USING GIST (geometry);
23
24 -- Detection results
25 CREATE TABLE detection_logs (
26     id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
27     grid_id     UUID NOT NULL REFERENCES grid_cells(id),
28     imagery_id  UUID REFERENCES satellite_imagery(id),
29     confidence  NUMERIC(5, 4) NOT NULL,
30     damage_category damage_category NOT NULL DEFAULT 'none',
31     bounding_boxes JSONB DEFAULT '[]',
32     annotated_path TEXT,
33     raw_image_path TEXT,
34     inference_time_ms INTEGER,
35     model_version VARCHAR(32) DEFAULT 'v1',
36     created_at  TIMESTAMPTZ DEFAULT NOW()
37 );
38
39 -- Auto-alert trigger
40 CREATE OR REPLACE FUNCTION auto_create_alert()
41 RETURNS TRIGGER AS $$
42 BEGIN
43     IF NEW.confidence >= 0.7
44     AND NEW.damage_category IN ('severe', 'moderate')
45     THEN
46         INSERT INTO alerts (detection_id, severity, summary)
47         VALUES (NEW.id,
48             CASE WHEN NEW.damage_category = 'severe'

```

```

49         THEN 'critical' ELSE 'high' END,
50         FORMAT('Deteksi s_pada_grid s_(%.1f%%)',
51             NEW.damage_category,
52             (SELECT grid_code FROM grid_cells
53              WHERE id = NEW.grid_id),
54             NEW.confidence * 100));
55     END IF;
56     RETURN NEW;
57 END;
58 $$ LANGUAGE plpgsql;
59
60 CREATE TRIGGER trg_auto_create_alert
61     AFTER INSERT ON detection_logs
62     FOR EACH ROW
63     WHEN (NEW.confidence >= 0.7)
64     EXECUTE FUNCTION auto_create_alert();

```

Query Contoh untuk Dashboard

Listing 2: Query untuk menampilkan grid di peta

```

1  -- Grid dalam bounding box (untuk map view)
2  SELECT id, grid_code,
3         ST_AsGeoJSON(geometry) AS geojson,
4         status, last_detection_id
5  FROM grid_cells
6  WHERE ST_Intersects(
7         geometry,
8         ST_MakeEnvelope(114.0, -3.0, 115.0, -2.0, 4326)
9  );
10
11 -- Statistik untuk sidebar dashboard
12 SELECT
13     status,
14     COUNT(*) AS grid_count,
15     SUM(area_ha) AS total_area_ha,
16     ROUND(SUM(area_ha) * 100.0 /
17            SUM(SUM(area_ha)) OVER(), 2) AS percentage
18 FROM grid_cells
19 WHERE status != 'unknown'
20 GROUP BY status;
21
22 -- Alert yang belum terkirim (polling WA Bot)
23 SELECT a.id, a.severity, a.summary,
24        dl.confidence, dl.damage_category,
25        gc.grid_code,
26        ST_X(gc.centroid::geometry) AS lat,
27        ST_Y(gc.centroid::geometry) AS lng,
28        dl.annotated_path
29 FROM alerts a
30 JOIN detection_logs dl ON a.detection_id = dl.id
31 JOIN grid_cells gc ON dl.grid_id = gc.id
32 WHERE a.notified = FALSE
33 ORDER BY a.created_at ASC LIMIT 10;

```

User Interface & User Experience

Dashboard Layout

- **Map Area (utama)** — Leaflet.js map dengan base layer OpenStreetMap. Overlay grid cells dengan warna:
 - **Hijau** — Tidak ada kerusakan (confidence < 0.3)
 - **Kuning** — Kerusakan ringan ($0.3 \leq \text{confidence} < 0.7$)
 - **Merah** — Kerusakan parah (confidence ≥ 0.7)
 - Abu-abu — Belum diproses / unknown
- **Sidebar Panel** — Statistik ringkasan, daftar recent alerts, filter kontrol, dan search box.
- **Detail Modal** — Muncul saat grid di-click. Menampilkan side-by-side raw vs annotated image, confidence score, bounding box details, timeline chart, dan action buttons.

Wireframe (Deskripsi)

1. **Default View:** Peta Indonesia dengan grid overlay. Semua grid berwarna hijau/abu-abu. Sidebar menunjukkan "No active alerts. Semua area dalam kondisi aman."
2. **Alert Scenario:** Tiba-tiba beberapa grid di Kalimantan berubah menjadi merah. Sidebar memunculkan alert card baru dengan animasi. WebSocket push notifikasi real-time. WA Bot mengirim pesan otomatis ke nomor terdaftar.
3. **Investigation:** Petugas klik grid merah. Modal detail muncul: perbandingan citra Landsat vs annotated image dengan bounding box di area yang terdeteksi gundul. Confidence: 94.3%. Timeline chart menunjukkan tren peningkatan deforestasi dalam 2 minggu terakhir.
4. **Action:** Petugas klik "Acknowledge" dan "Laporkan". Status alert berubah. Data otomatis tercatat untuk audit trail.

Format Notifikasi WhatsApp

PERINGATAN DINI DEFORESTASI

Lokasi:

Lat: -3.4567, Lng: 114.9876

Grid ID: GRID-2024-A1

Tingkat Kerusakan: SEVERE

Confidence: 94.3%

Waktu: 2024-01-15 14:30:22

<https://maps.google.com/?q=-3.4567,114.9876>

<https://deforest.id/grid/GRID-2024-A1>

[gambar annotated overlay dilampirkan]

Roadmap Pengembangan

Fase	Aktivitas	Deliverable
F0: Setup (Hari 1–2)	Instalasi Proxmox, Docker, NFS. Setup monorepo, struktur service, CI/CD dasar.	Semua container bisa start. <code>docker-compose up -d</code> running.
F1: Data (Hari 3–4)	GEE auth, fetcher, grid generator, image cropper, Redis queue integration.	Citra satelit sukses ditarik. Grid tersimpan di PostGIS. Queue trigger ke ML.
F2: ML (Hari 5–7)	YOLOv8 setup, preprocessing, inference worker, postprocessing (NMS), annotated image gen, alert trigger.	ML container bisa infer grid. Hasil di DB. Alert auto-created.
F3: API (Hari 5–7)	Bun + Elysia setup, REST endpoints, WebSocket, Redis cache, API docs.	Semua endpoint berfungsi. WebSocket push real-time.
F4: Bot (Hari 7–8)	Baileys setup, QR auth, alert polling, message format, image send, retry logic.	Bot kirim notifikasi < 60 detik dari deteksi.
F5: UI (Hari 8–10)	React + Vite + Leaflet setup, map overlay, sidebar, detail modal, WS client, timeline chart.	Dashboard aksesibel via browser. Grid warna real-time.
F6: Test (Hari 10–11)	Unit test, integration test, E2E flow test, security audit, load test.	Coverage > 70%. Semua flow end-to-end verified.
F7: Demo (Hari 12)	Final deploy, seed demo data, prepare demo script, slide deck, backup VM.	Sistem live siap demo. Slide presentasi siap.

Risk Management

Risiko	Dampak	Prob.	Mitigasi
GEE API rate limit	Data gagal ditarik	Rendah	Cache data, interval pull, fallback dummy data
ML akurasi rendah	False positive/negatif	Sedang	Fine-tune dataset lokal, ensemble model, human-in-the-loop
WA nomor diblokir	Notifikasi gagal	Rendah	Multi-session, cadangan SMS/email, retry queue
Container crash	Service down	Rendah	restart: always, healthcheck, resource limits
Proxmox total down	Blackout	Sangat Rendah	Snapshot rutin, backup DB cloud, migrasi plan
Dataset deforestasi Indonesia terbatas	Training suboptimal	Sedang	Pre-trained model + few-shot, atau rule-based fallback

Risiko	Dampak	Prob.	Mitigasi
Waktu hackathon terbatas	Tidak semua fitur selesai	Sedang	Prioritaskan core flow. Fitur "nice to have" terakhir

Success Metrics

Metrik	Target	Cara Ukur
End-to-end pipeline time	< 15 menit	Logging timestamp tiap fase
Grid processing speed	> 10 grid/detik	ML inference logging
Dashboard load time	< 3 detik	Lighthouse / manual
WebSocket latency	< 500 ms	Client-side timing
Alert delivery time	< 60 detik	created_at vs sent_at
System uptime	> 99% selama demo	Docker healthcheck
Code quality	No any type, lint pass	TypeScript strict, ESLint

Rencana & Struktur Tim

Pembagian Peran

Peran	Jumlah	Tanggung Jawab
Lead Engineer / Backend	1 orang	Arsitektur sistem, Backend API (Bun), Database (PostGIS), WebSocket, Deployment
ML Engineer	1 orang	YOLOv8 pipeline, dataset preparation, model training/optimasi, TFLite export
Frontend Engineer	1 orang	Dashboard (React + Leaflet), WebSocket client, visualisasi D3.js
DevOps / Integration	1 orang	Docker, Proxmox, CI/CD, GEE integration, WA Bot, Testing

Tim ideal terdiri dari **3–4 orang** dengan overlap skill untuk mengantisipasi jika ada anggota yang harus fokus di area lain.

Potensi Pengembangan ke Depan

- Multi-Satellite Fusion** — Integrasi data SAR (Sentinel-1) untuk deteksi awan (cloud-penetrating).
- Time-Series Analysis** — Deteksi tren deforestasi jangka panjang dengan LSTM/Transformer untuk prediksi area berisiko.
- Mobile App** — Aplikasi Android native untuk petugas lapangan dengan GPS tracking dan foto geo-tagged.

4. **Crowdsourced Validation** — Masyarakat bisa mengirim foto verifikasi dari lokasi yang terdeteksi, menjadi dataset training tambahan.
5. **Integration with Government Systems** — API untuk Kementerian LHK, KLHK, dan Sistem Informasi Geospasial Nasional.
6. **Carbon Accounting** — Estimasi emisi karbon berdasarkan luas area deforestasi terdeteksi.

Penutup

Deforest.id bukan sekadar alat monitoring — ini adalah **sistem peringatan dini yang mengubah paradigma** dari "menunggu laporan" menjadi "mengetahui sebelum terlambat." Dengan memanfaatkan teknologi satelit, machine learning, dan komunikasi massal (WhatsApp), sistem ini menghadirkan solusi yang **tepat, cepat, dan mudah diakses** oleh semua pemangku kepentingan — dari pejabat KLHK hingga petugas lapangan di pelosok Kalimantan.

*"Hutan Indonesia adalah paru-paru dunia.
Deforest.id adalah sistem saraf yang menjaganya tetap hidup."*

Kontak Tim:

Nama Tim: [Nama Tim]

Anggota: [Nama 1], [Nama 2], [Nama 3], [Nama 4]

Email: [email@example.com]

GitHub: github.com/username/deforest.id